

18. Алгоритми и структури за търсене. Търсене в масив. Обикновени и балансирани Двоични дървета за търсене (напр. Аделсон-Велский и Ландис дървета) - свойства, нарастване и височина.

1. Задачата за търсене

Търсенето е една от най-фундаменталните задачи в информатиката: даден е набор от елементи (записи) с ключове и трябва да се установи дали конкретен ключ k присъства в него, и ако да - да се намери свързаният с него запис (или позицията му). Начинът, по който данните са организирани определя пряко колко бързо може да се извърши търсенето, вмъкването и изтриването на елементи. Има 2 основни подхода:

1. Търсене в масив - данните са в непрекъсната линейна памет. Разглеждаме линейно и двоично търсене.
2. Търсене в дървовидна структура - данните са организирани в двоично дърво за търсете (BST), при което всеки връх позволява стесняване на областта на търсене. Разглеждаме обикновени BST и техния балансиран вариант - AVL дървета.

Основният критерий за сравнение е времевата сложност на операциите търсене, вмъкване и изтриване като функция от броя елементи n . При дървовидните структури - като функция от височината h на дървото.

2. Търсене в масив

2.1 Линейно търсене

При линейното търсене елементите се обхождат един по един, докато се намери търсеният ключ или се стигне до края на масива. Не се изисква масивът да е сортиран.

- Най-добър случай: $O(1)$ (елементът е на първа позиция).
- Най-лош и среден случай: $O(n)$.

```
for (int i = 0; i < size ; i++)  
    if (a[i] == key) return i;
```

2.2 Двоично търсене в сортиран масив

Ако масивът е сортиран, може да се приложи двоично търсене: сравняваме търсения ключ със средния елемент и продължаваме рекурсивно само в лявата или дясната половина.

```
int binarySearch(const int* a, int key) {  
    int lo = 0;  
    int hi = size - 1;  
  
    while (lo <= hi) {  
        int mid = lo + (hi - lo) / 2;  
        if (a[mid] == key) return mid;  
        else if (a[mid] < key) lo = mid + 1;  
        else hi = mid - 1;  
    }  
    return -1;  
}
```

Процесът на двоично търсене е подобен на слизване по балансирано двоично дърво, изградено от индексите на масива.

Всеки алгоритъм за търсене чрез сравнения в най-лошия случай изисква $\Omega(\log n)$ сравнения. Двоичното търсене достига тази граница и затова е **асимптотично оптимално**.

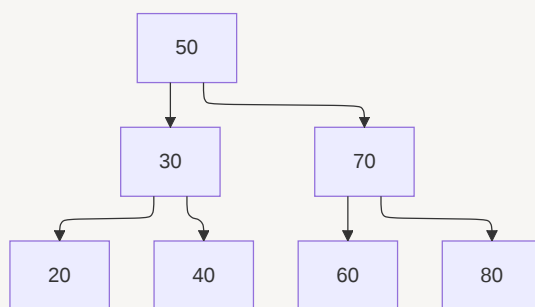
3. Обикновени двоични дървета за търсене (BST)

3.1 Определяне и BST-свойство

Двоично дърво за търсене е двоично дърво, в което всеки връх x съхранява ключ $x.key$ и удовлетворява следното свойство за подредба:

$$\forall y \in \text{ляво поддърво}(x) : y.key < x.key \text{ и } \forall z \in \text{дясно поддърво}(x) : z.key > x.key$$

т.е. всички ключове в ляво поддърво са по-малки от ключа на върха, а всички в дясно поддърво - по-големи. Свойството е **рекурсивно** - важи за **всеки връх**, а не само за корена.



3.2 Обхождане в нареден ред (in-order)

Обхождането става в реда ляво $\rightarrow$ връх $\rightarrow$ дясно, като така посещава ключовете в строго растящ ред. Точно затова структурата се нарича "дърво за търсене" - тя реализира сортиран ред без нужда от изричното пренареждане на елементите, характерно за масив.

3.3 Основни операции - търсене, вмъкване и изтриване

Търсене. Започвайки от корена, на всяка стъпка сравняваме търсения ключ с ключа на текущия връх и продължаваме наляво или надясно, докато намерим ключа или стигнем до `nullptr`.

```
struct Node {  
    int key;  
    Node* left = nullptr;  
    Node* right = nullptr;  
    Node(int k) : key(k){}  
};
```

```
Node* search(Node* root, int key) {
    if(!root || root->key == key) return root;
    return key < root->key ? search(root->left, key) : search(root->right);
}
```

Вмъкване. Търсим ключа, докато стигнем до липсващото дете, и там създаваме нов връх.

```
Node* insert(Node* root, int key) {
    if(!root) return new Node(key);
    if(key < root->key) root->left = insert(root->left, key);
    else if(key > root->key) root->right = insert(root->right, key);
    return root;
}
```

Изтриване. Има три случая за върха x , който трябва да се премахне.

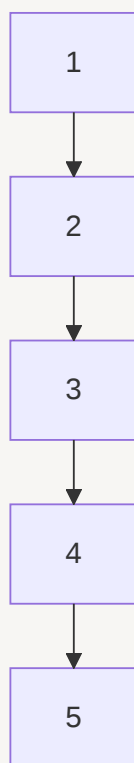
1. x е листо - просто се премахва
2. x има само едно дете - детето заема мястото на x
3. x има 2 деца - заменя се със своя наследник (мин. елемент в дясното поддърво, или макс. елемент в лявото поддърво), след което наследникът се изтрива.

3.4 Височина на BST и сложност на операциите

Всички три операции извършват работа, пропорционална на дължината на пътя от корена до търсения/вмъкваният/изтриваният връх - т.е. сложността им е $O(h)$, където h е височината на дървото.

Проблемът е, че височината зависи изцяло от **реда на вмъкване** на елементите:

- Ако елементите се вмъкват в "случаен" ред, очакваната височина е $O(\log n)$.
- В най-лошия случай - например на вече сортирана последователност 1, 2, 3, 4, 5 - дървото се превръща във верига, където всеки връх има само дясно дете.



При това дърво $h = n - 1$, следователно търсене/вмъкване/изтриване имат сложност $O(n)$ - колкото е линейно търсене в масив. Именно този недостатък мотивира въвеждането на балансирани дървета за търсене, при които височината е гарантирано $O(\log n)$ независимо от реда на вмъкванията.

4. Балансирани двоични дървета за търсене - AVL дървета

4.1 Мотивация и определение

AVL дървото е BST, в което за всеки връх x разликата между височините на лявото и дясното му поддърво е ограничена:

$$BF(x) = h(\text{дясно}(x)) - h(\text{ляво}(x)) \in -1, 0, 1$$

Това е локално условие (**важи за всеки връх поотделно**), но неговото поддържане гарантира глобален резултат - логаритмична височина на цялото дърво.

4.2 Височина на AVL дърво

Нека $N(h)$ е минималният възможен брой върхове в AVL дърво с височина h (височина на празно дърво приемаме за -1 , а на единичен връх -0).

За да е дървото с височина h минимално по брой върхове при спазване на AVL-свойството, едно от поддърветата на корена трябва да е с височина $h - 1$, а другото - с височина $h - 2$ (максимално допустимата разлика):

$$N(h) = 1 + N(h - 1) + N(h - 2), N(-1) = 0, N(0) = 1$$

Това е рекурентност от тип "Фибоначи".

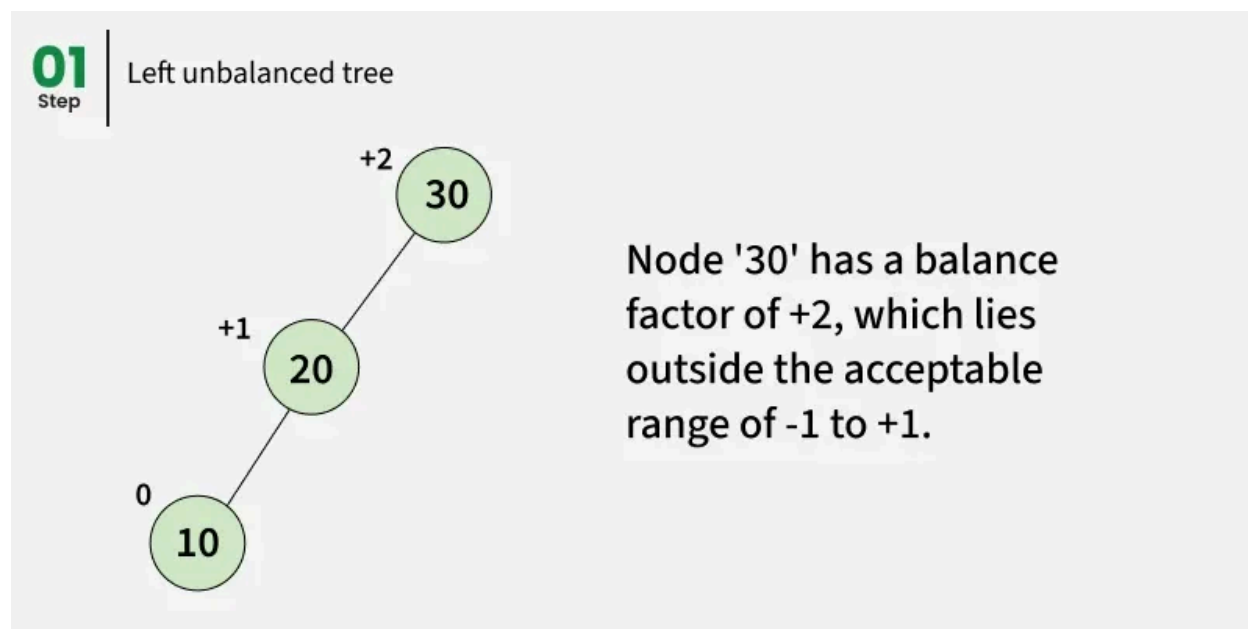
За произволно AVL дърво с n елемента и височина h , $h = O(\log_2 n)$. Височината е гарантирано логаритмична спрямо броя на елементите.

4.3 Възстановяване на баланса чрез ротации

След вмъкване или изтриване на връх AVL-свойството може да се нарушава.

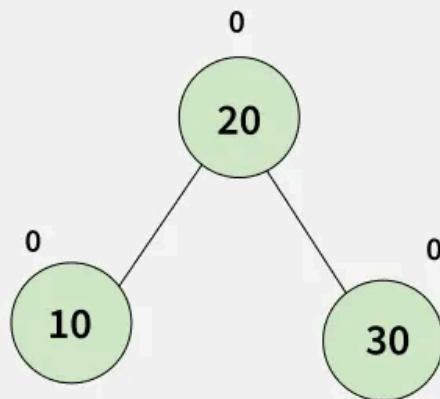
Нарушението се отстранява чрез една или две ротации, извършени в първия (най-нисък) нарушен връх по пътя нагоре. Различават се **четири случая**:

LL случай (единична дясна ротация) - дисбалансът е в лявото поддърво на лявото дете:



03
Step

Balanced tree

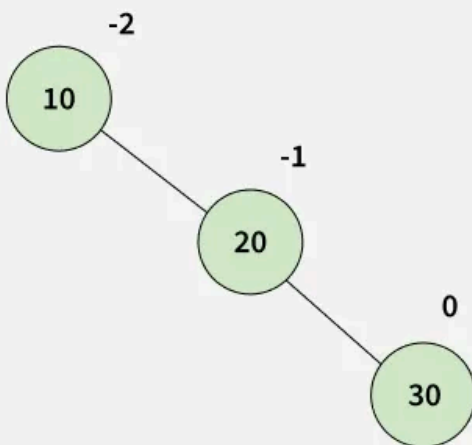


The tree is balanced now, with all node balance factors within the valid range.

RR случай (единична лява ротация) - дисбалансът е в дясното поддърво на дясното дете:

01
Step

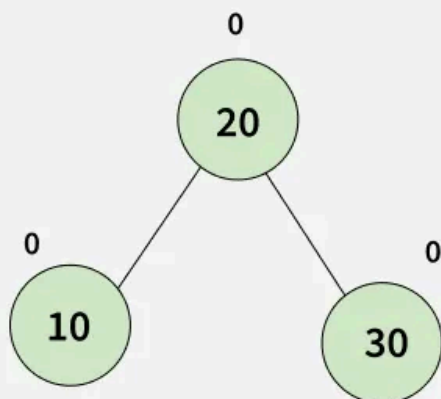
Right unbalanced tree



Node '10' has a balance factor of -2, which lies outside the acceptable range of -1 to +1.

03 Step

Balanced tree



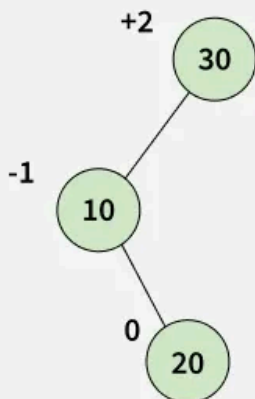
The tree is balanced now, with all node balance factors within the valid range.

LR и **RL** са комбинация от две единични ротации (съответно ляво-дясно и дясно-ляво) и се прилагат, когато дисбалансът е в "по-навътрешното" поддърво.

LR (Обърни внимание на това дали е ляво или дясно поддърво):

01 Step

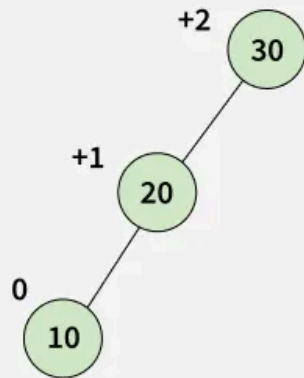
Unbalanced tree



Node '30' has a balance factor of +2, which lies outside the acceptable range of -1 to +1.

03
Step

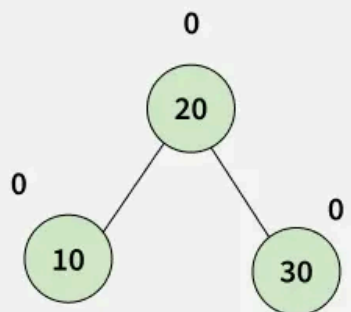
Unbalanced tree



we need to perform
right rotation on
nodes 30 and 20 to
balanced the tree

05
Step

Balanced tree

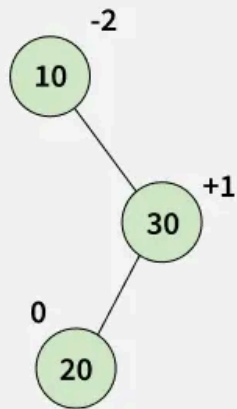


The tree is balanced
now, with all node
balance factors within
the valid range.

RL (Обърни внимание на това дали е ляво или дясно поддърво):

01
Step

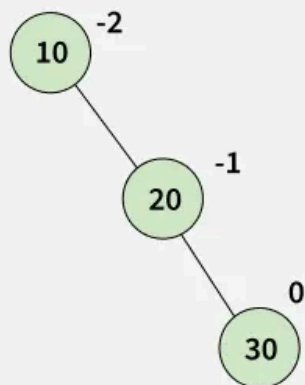
Unbalanced tree



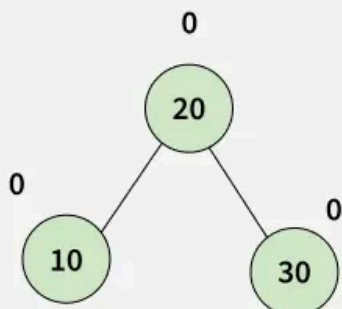
Node '10' has a balance factor of -2, which lies outside the acceptable range of -1 to +1.

03
Step

Unbalanced tree



we need to perform left rotation on nodes 10 and 20 to balanced the tree



The tree is balanced now, with all node balance factors within the valid range.

Всяка ротация е с константна сложност $O(1)$ и запазва BST-свойството (относителния ред на ключовете), докато намалява разликата във височините на поддърветата.

4.4 Вмъкване и изтриване в AVL дърво

- **Вмъкване:** извършва се обикновено BST-вмъкване (рекурсивно до липсващо дете), след което по пътя обратно към корена се обновява височината на всеки връх и се проверява балансовият му фактор. При първото нарушение (когато абс. стойност на балансовият фактор стане = 2) се прилага подходящата от четирите ротации.
- **Изтриване:** извършва се обикновено BST-изтриване, след което - аналогично на вмъкването - се обновяват височините и балансовите фактори по пътя нагоре, с ротации при нужда. Разлика спрямо вмъкването: при изтриване може да се наложи повече от една ротация по различни нива на пътя нагоре (при вмъкване е достатъчна само една).

4.5 Сложност на операциите в AVL дърво

Тъй като височината на AVL дървото е гарантирано $O(\log n)$.

Търсене, вмъкване, изтриване в AVL дърво = $O(\log n)$ в най — лошия случай.

Това е съществено подобрение спрямо обикновените BST, при които същите операции имат само средна сложност $O(\log n)$, но гарантирана горна граница $O(n)$

5. Сравнение на структурите за търсене

Структура	Търсене	Вмъкване	Изтриване
Несортиран масив	$O(n)$	$O(1)$	$O(n)$
Сортиран масив + двоично търсене	$O(\log n)$	$O(n)$ (пренареждане)	$O(n)$
Обикновено BST	$O(\log n)$ в средния случай; $O(n)$ в най-лошия	$O(\log n)$ в средния случай; $O(n)$ в най-лошия	$O(\log n)$ в средния случай; $O(n)$ в най-лошия
АВЛ дърво	$O(\log n)$	$O(\log n)$	$O(\log n)$